

CO3-AT1-DEBUGGING

NAME: V Rakshana

SLOT: B

REGNO :192425137

COURSE CODE: CSA0911

QUESTION-1:

CODE:

```
public class SynchronizedCounterDemo {  
    private int count = 0;  
    public synchronized void increment() {  
        count++;}  
    public synchronized void decrement() {  
        count = count - 1; }  
    public static void main(String[] args) throws InterruptedException {  
        SynchronizedCounterDemo demo = new SynchronizedCounterDemo();  
        Runnable task = () -> {  
            for (int i = 0; i < 1000; i++) {  
                demo.increment();}  
            }  
        Thread t1 = new Thread(task);  
        Thread t2 = new Thread(task);  
        t1.start();  
        t2.start();  
        t1.join();  
        t2.join();  
        System.out.println("Final count: " + demo.count);  
        synchronized (demo) {  
            System.out.println("Count inside synchronized block: " + demo.count);}  
        int finalCount = demo.count;  
        System.out.println("Final Count: " + finalCount);  
    }  
}
```

OUTPUT:

Your Code

```
1 public class SynchronizedCounterDemo {
2     private int count = 0;
3     public synchronized void increment() {
4         count++;
5     }
6     public synchronized void decrement() {
7         count = count - 1;
8     }
9     public static void main(String[] args) throws InterruptedException {
10         SynchronizedCounterDemo demo = new SynchronizedCounterDemo();
11         Runnable task = () -> {
12             for (int i = 0; i < 1000; i++) {
13                 demo.increment();
14             }
15         };
16         Thread t1 = new Thread(task);
17         Thread t2 = new Thread(task);
18         t1.start();
19         t2.start();
20         t1.join();
21         t2.join();
22         System.out.println("Final count: " + demo.count);
23         synchronized (demo) {
24             System.out.println("Count inside synchronized block: " + demo.count);
25         }
26         int finalCount = demo.count;
27         System.out.println("Final Count: " + finalCount);
28     }
29 }
```

Input

stdin input

Output

```
Final count: 2000
Count inside synchronized block:
2000
Final Count: .2000
```

QUESTION-2:

CODE:

```
public class ThreadBasics {

    public static void main(String[] args) {

        Thread t1 = new Thread(new MyTask());

        t1.start();

        MyTask task = new MyTask();

        Thread t2 = new Thread(task);

        t2.start();

        System.out.println("Thread name: " + t1.getName());

        try {

            t2.join();

        } catch (InterruptedException e) {

            System.out.println("Thread interrupted: " + e.getMessage());}}}

class MyTask implements Runnable {
```

@Override

```
public void run() {
```

```
    System.out.println("Task running on: " + Thread.currentThread().getName());}}
```

OUTPUT:

Your Code

```
1 public class ThreadBasics {
2     public static void main(String[] args) {
3         Thread t1 = new Thread(new MyTask());
4         t1.start();
5         MyTask task = new MyTask();
6         Thread t2 = new Thread(task);
7         t2.start();
8         System.out.println("Thread name: " + t1.getName());
9         try {
10            t2.join();
11        } catch (InterruptedException e) {
12            System.out.println("Thread interrupted: " + e.getMessage());
13        }
14    }
15 }
16 class MyTask implements Runnable {
17     @Override
18     public void run() {
19         System.out.println(
20             "Task running on: " + Thread.currentThread().getName()
21         );
22     }
23 }
```

Input

stdin input

Output

```
Task running on: Thread-0
Thread name: Thread-0
Task running on: Thread-1
```

QUESTION-3:

CODE:

```
public class FinallyReturnDemo {

    public static int riskyDivision(int a, int b) {

        try {

            int result = a / b;

            return result;

        } catch (ArithmeticException e) {

            return -1;

        } finally {

            return 0;}}

    public static void main(String[] args) {

        try {

            System.out.println("Result: " + riskyDivision(10, 0));

        } catch (ArithmeticException e) {
```

```

        System.out.println("Caught: " + e.getMessage());}

int x = 5;

try {

    x = 10;

    throw new RuntimeException("fail");

} catch (RuntimeException e) {

    System.out.println("Caught RuntimeException: " + e.getMessage());

} finally {

    x = 20;

    System.out.println("Value of x in finally: " + x);}

System.out.println("Final value of x: " + x);

int y = 0;

try {

    y = compute();

} finally {

    System.out.println("Finally block executed."); }

System.out.println("Value of y: " + y);}

static int compute() {

    return 42;}}

```

OUTPUT:

Your Code

```

1 public class Main {
2     public static void main(String[] args) {
3         int[] numbers = {10, 20, 30, 40, 50};
4         SumThread sumThread = new SumThread(numbers);
5         AverageThread averageThread = new AverageThread(numbers);
6         MaximumThread maximumThread = new MaximumThread(numbers);
7         sumThread.start();
8         averageThread.start();
9         maximumThread.start();
10        try {
11            sumThread.join();
12            averageThread.join();
13            maximumThread.join();
14        } catch (InterruptedException e) {
15            System.out.println("Thread interrupted");
16        }
17        System.out.println("All calculations completed.");
18    }
19 }
20 class SumThread extends Thread {
21     int[] numbers;
22     SumThread(int[] numbers) {
23         this.numbers = numbers;
24     }
25     public void run() {
26         int sum = 0;

```

Input

stdin input

Output

```

Maximum = 50
Sum = 150
Average = 30.0
All calculations completed.

```